

Documentation of FastPose implementation

Mayank Yadav

January 20, 2026

Abstract

This report documents the implementation of the FastPose architecture for spacecraft pose estimation. It details the adaptation of the dataset, the integration of object detection, the geometric parameterization of the pose regression, and an analysis of the experimental results on the SPEED dataset.

1 Introduction to the dataset

1.1 Datasets

We use two main datasets in this project:

1. **SPEED**: Contains 12 000 synthetic images and 5 real images with pose labels in quaternion form. Images are 8-bit monochrome JPEGs at a resolution of 1920×1200 pixels. Note that images in the Test split do not have corresponding labels.
2. **SPEED+**: Similar to SPEED but with a larger number of images and more real images with labels.

Table 1: Split statistics for SPEED and SPEED+.

splits	SPEED		SPEED+		
	synthetic	real	synthetic	lightbox	sunlamp
Train	12000	5	47966	—	—
Validation	—	—	11994	—	—
Test	2998	300	—	6740	2791

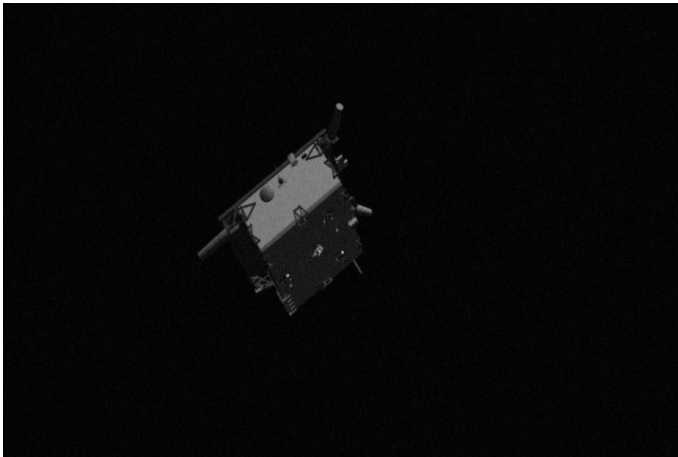


Figure 1: One Dataset Example: left — Raw satellite image; right — Corresponding pose-label.

1.2 Which dataset am I currently using?

I am currently using the SPEED dataset for training and evaluation of the FastPose model.

1.2.1 Training and validation split

Out of the 12 000 labeled synthetic images, 9 600 are used for training and 2 400 for validation. After the model is trained, the translation error and orientation error are computed on this validation split and on the five real images.

2 How Object detection and Pose estimation are integrated?

2.1 Independent cropping via projected model keypoints

To avoid a costly two-pass pipeline the detector is not used to produce the crop fed to FastPose. Instead we compute a tight image crop from the 3D model keypoints (the wireframe model can be visualised at <https://www.desmos.com/3d/jud6lmg9gn>) projected into the image using the current camera intrinsics and pose.

Expand the box by a small padding factor, clamp to image bounds, then crop and resize the resulting patch to the FastPose input resolution. This single crop is passed directly to the FastPose network for pose refinement; no detector-generated crop is required.

3 Object Detection Module

The object detection module is responsible for localizing the spacecraft within the image to provide a cropped region of interest for the pose estimation network. Since the SPEED dataset provides 6-DOF pose labels rather than explicit bounding box annotations, we implement a custom pipeline to generate ground truth boxes and train a lightweight detection transformer.

3.1 Ground Truth Generation via Projection

We generate bounding boxes by projecting the known 3D wireframe of the satellite model onto the 2D image plane using the camera’s geometric properties.

This process is implemented in the `project_keypoints` function. The respective Python functions are taken from [T.Park’s GitHub repository](#). Given a set of sparse 3D model vertices $\mathbf{X}_w \in \mathbb{R}^3$, the corresponding 2D pixel coordinates $\mathbf{u} \in \mathbb{R}^2$ are computed via the following steps:

1. **Rigid Body Transformation:** The vertices are transformed from the model frame to the camera coordinate system using the rotation matrix $\mathbf{R}$ (derived from $\mathbf{q}$) and the position vector $\mathbf{t}$:

$$\mathbf{X}_{cam} = \mathbf{R}(\mathbf{q}) \cdot \mathbf{X}_w + \mathbf{t} \quad (1)$$

2. **Perspective Projection & Distortion:** The camera coordinates $\mathbf{X}_{cam} = [x, y, z]^T$ are normalized and distorted to account for lens curvature (radial and tangential), denoted by the function $\mathcal{D}(\cdot)$:

$$\mathbf{x}_n = \mathcal{D}\left(\frac{x}{z}, \frac{y}{z}\right) \quad (2)$$

3. **Pixel Mapping:** Finally, the coordinates are mapped to the image plane using the camera intrinsic matrix $\mathbf{K}$:

$$\mathbf{u} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} \cdot \begin{bmatrix} \mathbf{x}_n \\ 1 \end{bmatrix} \quad (3)$$

The bounding box B_{gt} is then defined by the minimum and maximum u, v coordinates of the projected keypoints., expanded by a 5% margin to ensure the entire satellite is captured:

$$B_{gt} = [\min(x), \min(y), \max(x), \max(y)] \pm \text{margin} \quad (4)$$

3.2 Network Architecture: LW-DETR

We utilize a custom architecture named **LW_Detr** (Lightweight Detection Transformer). Unlike traditional detectors that rely on anchors and Non-Maximum Suppression (NMS), this model predicts bounding box coordinates directly using an attention mechanism.

The architecture consists of three stages:

1. **CNN Backbone:** We use a ResNet-18 pre-trained on ImageNet. We discard the final pooling and fully connected layers, projecting the features from 512 channels down to a dimension of $d_{model} = 256$.
2. **Transformer Encoder-Decoder:** The flattened feature maps are combined with learnable positional embeddings (`row_embed`, `col_embed`) to preserve spatial information. These are processed by a standard Transformer. A single learnable object query (`query_pos`) is used to attend to the salient object in the image.
3. **Prediction Head:** A Multi-Layer Perceptron (MLP) takes the transformer output and predicts the normalized box coordinates $[c_x, c_y, w, h]$.

3.3 Loss Function

To train the regressor, we employ a linear combination of the L_1 distance loss and the Generalized Intersection over Union (GIoU) loss. The GIoU loss is critical as it provides meaningful gradients even when the predicted box and ground truth box do not overlap.

$$\mathcal{L}_{total} = \lambda_{L1} \cdot \|B_{pred} - B_{gt}\|_1 + \lambda_{GIoU} \cdot \mathcal{L}_{GIoU}(B_{pred}, B_{gt}) \quad (5)$$

In our implementation, we set the hyperparameters $\lambda_{L1} = 5.0$ and $\lambda_{GIoU} = 2.0$ to balance the magnitude of the two terms.

3.4 Results Summary

Table 2 presents the quantitative evaluation of the object detection module after 70 epochs of training. We report the Mean and Median Intersection over Union (IoU) scores across both the synthetic and real test splits of the SPEED dataset. High IoU scores indicate that the model reliably crops the satellite, providing a stable input for the subsequent pose estimation stage.

Table 2: Object Detection Performance: IoU metrics on SPEED synthetic and real test-sets.

Metric	SPEED synthetic test-set	SPEED real test-set
Mean IoU (-)	0.8817	0.8583
Median IoU (-)	0.9134	0.8219

4 FastPose Pose Estimation Module

The FastPose module is a regression-based network designed to refine the 6-DOF pose of the spacecraft given a cropped image patch. Unlike the detection module which predicts bounding boxes, FastPose directly regresses the relative translation and rotation parameters.

4.1 Data Preprocessing and Augmentation

To bridge the domain gap between synthetic training data and real imagery, and to ensure geometric consistency, we implement the given rigorous preprocessing and augmentation pipeline given in the research paper.

4.1.1 Crop Generation

Before augmentation, the target bounding box is processed to ensure the network receives a consistent aspect ratio. We apply a padding margin of 15% to the tight bounding box derived from the 2D keypoint projections. Crucially, the bounding box is *squared* by expanding the shorter dimension to match the longer dimension. This prevents aspect ratio distortion when resizing the crop to the network input resolution of 384×384 . If the crop extends beyond image boundaries, we pad the region with zeros (black), preserving the center of the object.

4.1.2 Spatial Augmentation

Spatial augmentations modify both the input image and the ground truth pose labels (q, t) dynamically:

- **In-plane Rotation:** With a probability of 0.5, the image is rotated. The rotation angle θ is sampled from two regimes: small perturbations $\theta \in [-20^\circ, 20^\circ]$ (50% chance) or large inversions $\theta \in [160^\circ, 200^\circ]$ (50% chance). The ground truth translation and quaternion are updated via the rotation matrix $R_z(\theta)$.
- **Bounding Box Jitter:** The crop region is randomly perturbed by scaling the width and height independently by a factor $\in [-10\%, +10\%]$. This forces the network to be robust to imperfect object detection crops.

4.1.3 Pixel-level Augmentation

We utilize the `Albumentations` library to apply photometric distortions. These are applied stochastically with a probability of 0.5 for each group:

- **Color Jitter:** Random shifts in RGB values, or changes to brightness, contrast, hue, and saturation.
- **Blur and Noise:** Application of Motion Blur, Gaussian Blur, or Gaussian Noise injection to simulate sensor degradation.
- **Compression Artifacts:** Random JPEG compression (quality 85–100) to simulate transmission artifacts.

4.2 Pose Parameterization

The actual ingenuity of this research paper lies here. Instead of directly prediction the quaternion orientation, they are predicting the apparent orientation. They observed that when a cropped image is used, the network can better learn the relative pose with respect to the camera frame rather than the absolute pose in the world frame. That’s why they are making the network predict the apparent orientation and normalised translation.

They gave the example that when a cube at the centre is just translated to one side of the image, its apparent orientation changes due to perspective distortion. So, predicting the apparent orientation helps the network to better learn the pose from cropped images which are then mapped back to the full image scale using deterministic transformations.

All the respective python functions used are obtained from Pierre Victor Ancey by emailing him.

4.2.1 Apparent Rotation and 6D Representation

Due to perspective distortion, an object’s perceived orientation changes with its position in the image frame. To decouple this effect, the network predicts the *apparent orientation* $R' \in SO(3)$, defined as the orientation a centered object would require to visually match the translated object’s appearance.

We utilize a continuous 6D rotation representation, regressing the first two columns of the apparent rotation matrix, $\hat{r}_1, \hat{r}_2 \in \mathbb{R}^3$. The orthonormal apparent rotation matrix R' is recovered via the Gram-Schmidt process:

$$r_1 = \frac{\hat{r}_1}{\|\hat{r}_1\|}, \quad r_2 = \frac{\hat{r}_2 - (r_1 \cdot \hat{r}_2)r_1}{\|\hat{r}_2 - (r_1 \cdot \hat{r}_2)r_1\|}, \quad r_3 = r_1 \times r_2 \quad (6)$$

where $R'_{pred} = [r_1, r_2, r_3]$. The true global rotation R is subsequently recovered by applying a corrective rotation ΔR , derived from the object’s translation direction, such that $R = \Delta R^\top R'_{pred}$.

4.2.2 Translation via Normalized Coordinates

To handle the scale ambiguity introduced by cropping, we regress a normalized translation vector $U = [U_x, U_y, U_z]$. The lateral components (U_x, U_y) represent the offset between the projected object center and the crop center (b_x, b_y) , normalized by the crop dimensions (w, h) :

$$U_x = \frac{1}{w} \left(\frac{f_x X}{Z} + c_x - b_x \right), \quad U_y = \frac{1}{h} \left(\frac{f_y Y}{Z} + c_y - b_y \right) \quad (7)$$

The depth component U_z is formulated as a ‘‘crop-depth proxy,’’ related to the true depth Z via scaling factors s_x and s_y (derived from the ratio of full image to crop dimensions):

$$U_z = \frac{1}{2} \left(\frac{1}{s_x} + \frac{1}{s_y} \right) Z \quad (8)$$

This parameterization ensures the regression targets remain invariant to crop scale and camera intrinsics.

4.3 Network Architecture

We employ a Vision Transformer (ViT) architecture as the backbone. Specifically, we use the ViT-B/16 model pre-trained on ImageNet with SWAG weights (IMAGENET1K_SWAG_E2E_V1).

The network accepts a fixed input resolution of 384×384 pixels. The standard classification head is replaced with a single linear layer that projects the 768-dimensional latent feature vector to a 9-dimensional output vector:

$$y_{pred} \in \mathbb{R}^9 = \{t_{rel} \in \mathbb{R}^3, r_{6d} \in \mathbb{R}^6\} \quad (9)$$

4.4 Loss Function

The network is trained to minimize a combined loss function consisting of a translation term and a rotation term. We utilize the Mean Squared Error (MSE) for both components:

$$\mathcal{L}_{total} = \mathcal{L}_{rot} + \mathcal{L}_{trans} \quad (10)$$

The rotation loss is calculated as the MSE between the predicted 6D vector and the ground truth 6D representation (effectively the Frobenius norm of the difference between the first two columns of the rotation matrices). The translation loss is the MSE between the predicted relative translation U_{pred} and the ground truth U_{gt} .

4.5 Optimization Strategy

The model is trained using a hybrid optimizer approach to ensure stability in the deep Transformer backbone:

- **Muon (Momentum Orthogonalized by Newton-Schulz):** Used for all hidden 2D weights (matrix multiplications). This optimizer performs an orthogonalization post-processing step using 5 Newton-Schulz iterations to keep the spectral norm of updates constrained.
- **AdamW:** Used for embeddings, biases, normalization layers, and the final regression head.

We employ Automatic Mixed Precision (AMP) for memory efficiency. The learning rate follows a Cosine Annealing schedule with Warm Restarts ($T_0 = 10$, $T_{mult} = 2$).

4.6 Results Summary

We evaluate the performance of the FastPose module on both the SPEED synthetic test-set and the SPEED real test-set after 310 epochs. The evaluation metrics include the mean and median Translation Error (E_T) and Rotation Error (E_R), as well as the composite SPEED Score.

Table 3: Quantitative results on SPEED Synthetic and Real test sets.

Metric	SPEED Synthetic Test-set	SPEED Real Test-set
Mean E_T (m) vector	$[-0.000, 0.001, -0.005]$	$[-0.003, 0.012, 0.025]$
Mean $ E_T $ (m)	0.0046	0.0280
Median $ E_T $ (m)	0.032	0.044
Mean E_R (deg)	1.2238	4.7625
Median E_R (deg)	1.1163	3.3983
Speed Score	0.0257	0.0943

4.6.1 Comparison with the Results given in the research paper

Comparison with the Results reported in the paper The paper reports mean translation error $\overline{E_T} = 0.0221$ m and mean rotation error $\overline{E_R} = 0.4183^\circ$. Table below compares those values with our measured results.

Table 4: Paper vs. our results (mean errors).

Metric	Paper	Ours (SYN)	Ours (REAL)
Mean $ E_T $ (m)	0.0221	0.0046	0.0280
Mean E_R (deg)	0.4183	1.2238	4.7625

Our synthetic translation error is lower than the paper’s reported value, while our rotation errors (both synthetic and real) are larger; on the real set both translation and rotation exceed the paper’s reported performance.

4.6.2 Possible Reasons for Different Results

1. Directly using crop using keypoints instead of using detector output The translational error ($\overline{|E_T|} = 0.0046$ m) is significantly lower than the State-of-the-Art result reported in the paper (0.0221 m). This could be because we are directly using the crop obtained from the projected keypoints instead of using the detector output to get the crop. The detector might not be giving very accurate bounding boxes which could lead to higher translation error. By using the projected keypoints, we are providing a perfect crop to the FastPose network which is likely leading to lower translation error.

2. Training on less number of images We are training the FastPose network on only 9600 synthetic images whereas the paper might have trained on 12000 synthetic images and 5 real images. This could be a reason why our rotation error ($\overline{E_R} = 1.2238^\circ$ on synthetic and 4.7625° on real) is significantly higher than the paper’s reported rotation error (0.4183°).

3. Detrimental Augmentation on Synthetic Data The code applies pixel-level augmentations (blur, noise, compression) to the SPEED dataset via `is_augment=True`. However, the paper explicitly conducts an ablation study showing that for the SPEED dataset (where both train and test are synthetic), **pixel augmentations are detrimental**.

- **Paper:** "This can be explained by the fact that both the training and test images are primarily synthetic, so pixel-level distortions do not reflect the target distribution and instead hinder learning."
- **Code:** The `Albumentations` pipeline injects noise and blur, which likely hindered the model’s ability to learn clean geometric edges necessary for precise rotation estimation on the synthetic domain.